22.10.2026

Лекция 7. Функции, агрегация, подзапросы

Выражения и функции

При делении целого на целое получается целое.

Можно приводить типы через `CAST` или `::`. Например, `CAST(price AS INTEGER)` или `price::INTEGER`.

`ROUND` для округления и `NUMERIC` для точности у вещественных чисел (для денег, например).

Любая арифметика с `NULL` дает `NULL`. Сравнение с `NULL` дает `UNKNOWN`.

Конкатенация с `NULL` через `||` дает `NULL`. Конкатенация через `CONCAT` оставит на месте `NULL` пробел, а `CONCAT_WS` нет.

`NULLIF(a, b)` дает выдает `NULL`, если `a=b`. Используется для защиты от деления на ноль.

Даты можно хранить по разному: только дату, интервал, дату + время + часовой пояс. Даты нужно проверять на `NULL`.

`BETWEEN` нельзя использовать для `timestamp`.

Агрегатные функции

Агрегатная функция превращает набор строк в 1 значение.

Функция	**Что считает?**
COUNT	Число строк или значений
SUM	Сумму
AVG	Среднее
MIN	Минимум

Функция	Что считает?
MAX	Максимум

3 вида COUNT

1. `Count(*)` считает строки
2. `Count(col)` считает значения не равные `NULL`
3. `Count(DISTINCT col)` считает различные значения

Агрегаты и NULL

`SUM`, `AVG`, `MIN`, `MAX` игнорируют `NULL`.

`AVG(rating)` делит на число курьеров с рейтингом. `SUM(rating) / COUNT(*)` делит на все строки.

Сумма пустого множества дает `NULL`. `COALESCE(SUM(price), 0)` даст 0 для пустого набора.

FILTER

Есть только у `PostgreSQL`.

```
SELECT * FROM orders
COUNT(*) FILTER(WHERE status='formed');
```

В других СУБД для этого используют `CASE`.

Ловушки

Агрегат нельзя писать в `WHERE`. `WHERE` выполняется до группировки.

Агрегаты нельзя вкладывать друг в друга.

`SUM(DISTINCT price)` считает разничные значения в цены, а не различные товары. Обычно `DISTINCT` уместен внутри агрегата только в `COUNT`.

Группировка

`GROUP BY` разбивает строки на группы с одинаковыми значениями ключа группировки.

Каждый столбец `SELECT` должен быть либо в `GROUP BY`, либо внутри агрегата (кроме случая, когда группировка идет по первичному ключу).

```
SELECT status, total_amount, COUNT(*)
FROM orders
GROUP BY status; -- ошибка: total_amount не в агрегате и не в GROUP BY
```

Все NULL в группировке образуют единую группу.

WHERE и HAVING

	WHERE	**HAVING**
Что фильтрует?	Строки	Группы
Когда выполняется?	До группировки	После группировки
Можно ли писать агрегаты внутри?	Нельзя	Можно

Особенности

- Псевдоним из `SELECT` нельзя использовать в `HAVING`
- `HAVING` без `GROUP BY` считает всю таблицу единой группой
- `ORDER BY` может использовать пвсвдоним

Простые подзапросы

Подзапрос - запрос, который ни от чего не завит. Он выполняется один раз и его результат используется в другом запросе.

Скалярный подзапрос

```sql
SELECT product_id, name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Если скалярный подзапрос вернет больше 1 строки будет ошибка.

Подзапрос в IN

```sql
SELECT customer_id, last_name, first_name
FROM customers
WHERE customer_id IN (
     SELECT customer_id
     FROM orders
     WHERE status = 'delivered'
);
```

Есть не только `IN`, но и `ANY` и `ALL`. `ALL` по пустому множеству возвращает `TRUE`, а `ANY` вернет `FALSE`.

Подзапрос в FROM

```sql
SELECT orders_cnt, COUNT(*) as customers_cnt
FROM (
     SELECT customer_id, COUNT(*) AS orders_cnt FROM orders
)
GROUP BY customer_id;
```

Вложенная агрегация через подзапрос

Нельзя писать `AVG(COUNT(*))`. Вместо этого нужно писать так:

```sql
SELECT ROUND(AVG(orders_cnt), 2) AS avg_orders_per_customer
FROM (
     SELECT customer_id, COUNT(*) AS orders_cnt
     FROM orders
     GROUP BY customer_id
) AS t;
```

Подзапрос в HAVING

```
SELECT category, ROUND(AVG(price), 2) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > (SELECT AVG(price) FROM products)
ORDER BY avg_price DESC;
```

Порядок обработки запроса

Порядок записи

```
SELECT → DISTINCT → FROM → WHERE → GROUP BY → HAVING
> ORDER BY → LIMIT
```

Порядок выполнения

```
FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT
> ORDER BY → LIMIT
```